Pedal Pro Rentals

Agentforce AI Assistant — Master Project Reference

A complete, step-by-step record of everything built: data model, Flows, Agentforce agent configuration, the guardrail, every bug encountered and how it was fixed, the full testing log, and the remaining steps to publish this as a portfolio project.

Built by: Srinivas Sayinni | **Org:** Salesforce Developer Edition | **Platform:** Agentforce Studio

Table of Contents

- 1. Project Summary
- 2. Data Model — Objects and Fields
- 3. Seed Data Reference
- 4. Flow 1: Book_Rental_Flow (booking + bike status update)
- 5. Flow 2: Check_Bike_Availability (availability search)
- 6. Flow 3: Get_Booking_Name (booking lookup + 2-hour calculation)
- 7. Flow 4: Cancel_Booking (cancellation)
- 8. Flow 5: Escalate_To_Human (guardrail escalation to Case)
- 9. Agentforce Agent Structure (subagents, routing)
- 10. Final Agent Actions Reference (all 5, with exact I/O)
- 11. Final Reasoning Instructions (exact text, per subagent)
- 12. The Guardrail — How It Works
- 13. Bug Log — Every Issue Hit and How It Was Fixed
- 14. Permission Set — Agentforce Bike Access
- 15. Verified Test Log
- 16. What's Left To Do (Recording → GitHub → Resume)
- 17. Key Lessons Learned

1. Project Summary

Pedal Pro Rentals is a fictional bike rental business. This project is a custom Agentforce AI agent that helps customers check bike availability, book a rental, and cancel an existing booking — while enforcing one business guardrail: **a rental cannot be cancelled within 2 hours of its start time**; instead, the request is escalated to a human via a Salesforce Case.

Core capabilities demonstrated

- Grounded retrieval — agent answers using live Salesforce data, not invented information
- Flow-triggered record creation — booking creates a Rental__c record and updates Bike__c status in one step
- Record updates — cancellation updates Rental__c status; escalation creates a Case
- A guardrail — a real business rule enforced in agent instructions + Flow logic, not just a prompt suggestion

Tech stack

- Salesforce Developer Edition org
- 2 custom objects: Bike__c, Rental__c
- 5 Autolaunched Flows powering 5 custom Agentforce actions
- Agentforce Studio: 1 agent, 3 subagents (availability, booking, cancellation), router + escalation/off-topic/ambiguous defaults
- 1 custom Permission Set granting the agent's dedicated user access to Bike__c, Rental__c, and Case

2. Data Model — Objects and Fields

Bike__c

Field Label	API Name	Type	Notes
Bike Name	Name	Text(80)	Standard Name field
Model	Model__c	Text(80)	e.g. Trek Trail Blazer
Type	Type__c	Picklist	Mountain / Road / Hybrid / Electric — plain picklist, NOT multi-s
Location	Location__c	Text(120)	Free text, e.g. Gachibowli, Jubliee Hills
Status	Status__c	Picklist	Available / Rented / Maintenance — plain picklist, NOT multi-se
Hourly Rate	Hourly_Rate__c	Currency(6,2)	

Rental__c

Field Label	API Name	Type	Notes
Rental Name	Name	Text(80)	Standard Name field, entered manually per record
Bike	Bike__c	Lookup(Bike)	
Customer	Customer__c	Lookup(Contact)	Must point to the STANDARD Contact object — a common r
Start Date Time	Start_Date_Time__c	Date/Time	Salesforce auto-formatted the API name from Start_DateTim
End Date Time	End_Date_Time__c	Date/Time	Same auto-formatting note as above
Status	Status__c	Picklist	Booked / Active / Completed / Cancelled
Late Fee	Late_Fee__c	Currency(6,2)	Default 0

Note: When creating any picklist field, always double-check the Data Type radio button is set to plain **Picklist**, not **Picklist (Multi-Select)** — this project hit that exact mistake on both Type__c and Status__c and had to delete and recreate both fields, since the API name and multi-select setting cannot be edited after a field is saved.

3. Seed Data Reference

Sample Bike__c records created for testing (values may have changed status during testing due to bookings):

Model	Type	Location	Status (initial)	Rate
Trek Trail Blazer / Royal Enfield	Mountain	Jubliee Hills	Available	$160
Ducati Panigale	Hybrid	Gachibowli	Available	$100
TVS RR310	Road	Secundrabad	Available (later Rented in testing)	$60
KTM	—	—	Booked in guardrail test	—

Test Contact used throughout: **Contact ID 003g800000kcxMLAAY** ("srinivas Naidu")

Note: Record IDs are unique per org and will differ if this project is rebuilt from scratch in a new Developer Edition org. Always re-derive fresh IDs from the actual records rather than reusing the ones in this document.

4. Flow 1: Book_Rental_Flow

Type: Autolaunched Flow (No Trigger) | **Purpose:** Books a rental and updates the bike's status in a single call.

Input variables

- bikeId — Text — Available for input
- customerId — Text — Available for input
- startDateTimeText — Text — Available for input (see note on Date/Time below)
- endDateTimeText — Text — Available for input

Formula resources

- startDateTimeConverted — Formula, Date/Time — Formula: DATETIMEVALUE({!startDateTimeText})
- endDateTimeConverted — Formula, Date/Time — Formula: DATETIMEVALUE({!endDateTimeText})

Output variables

- success — Boolean — Available for output
- confirmationId — Text — Available for output
- resultMessage — Text — Available for output

Canvas logic

Start → **Get Bike** (Get Records: Bike__c where Id = bikeId AND Status__c = 'Available', store as first record) → **Bike Available?** (Decision) →

- **Available branch** (Get_Bike is not null): Create Rental (Create Records → Rental__c: Bike__c=bikeId, Customer__c=customerId, Start_Date_Time__c={!startDateTimeConverted}, End_Date_Time__c={!endDateTimeConverted}, Status__c='Booked') → Update Bike Status (Update Records → find Bike__c where Id={!Get_Bike.Id}, set Status__c='Rented') → Set Success Output (Assignment: success=True, confirmationId={!Create_Rental.Id}) → End
- **Not Available branch** (default outcome): Set Not Available Message (Assignment: resultMessage='Bike is not available') → End

Note: Both branches must independently connect to End. Do NOT let one branch's Assignment element feed into the other branch's Assignment — this was an actual bug hit during the build, where 'Set Not Available Message' was accidentally wired in series after 'Set Success Output', causing every successful booking to also overwrite the result with 'Bike is not available'.

Watch out: Date/Time input variables fed directly from Agentforce arrive typed as a generic 'Object', not a real Date/Time, and Flow execution fails silently when the agent (not a human via Debug) invokes it. The fix: take the date/time as plain **Text** input, then convert inside the Flow using a Formula resource with DATETIMEVALUE(). This was the single hardest bug in the whole build.

5. Flow 2: Check_Bike_Availability

Type: Autolaunched Flow | **Purpose:** Looks up bikes matching a type and location that are still Available.

Input variables

- bikeType — Text — Available for input
- location — Text — Available for input

Output variables

- resultMessage — Text — Available for output (simple found/not-found message)
- bikeCount, bikeListText — Text/Number — leftover unused outputs from earlier attempts; harmless if left in place

Canvas logic

Start → **Get Available Bikes** (Get Records: Bike__c where Type__c=bikeType AND Location__c=location AND Status__c='Available', store All records) → **Bikes Found?** (Decision: is Get_Available_Bikes null = False) →

- **Found branch:** Set Found Message (Assignment: resultMessage = 'Bikes are available matching your search.') → End
- **None Found branch** (default): Set None Found Message (Assignment: resultMessage = 'No bikes currently match your search.') → End

Watch out: An earlier version tried to build a formatted list of bike names using a Loop + Assignment. This repeatedly failed with UNKNOWN_EXCEPTION when called live by the agent (it worked fine in manual Debug). The final, reliable version avoids the Loop entirely and just returns a plain-text status message — simpler and far more stable. A future improvement would be to properly expose bike names/IDs as a structured output for the agent to relay, but that is out of scope for this build.

6. Flow 3: Get_Booking_Name

Type: Autolaunched Flow | **Purpose:** Looks up an existing booking and calculates how many minutes remain until its start time — this calculation is what powers the guardrail.

Note: This Flow's real name ended up being **Get_Booking_Name** (a naming slip during the build), not "Get_Booking_Details" as originally planned. The Agentforce action correctly references this exact name.

Input variables

- bookingId — Text — Available for input

Formula resources

- minutesUntilStart — Formula, Number — Formula: (Get_Booking.Start_Date_Time__c - NOW()) * 1440 (the *1440 converts a day-fraction difference into minutes)

Output variables

- bookingStartTime — Date/Time — Available for output
- bookingStatus — Text — Available for output
- bookingBikeId — Text — Available for output
- minutesUntilStartOutput — Number — Available for output

Canvas logic

Start → **Get Booking** (Get Records: Rental__c where Id=bookingId, store only the first record, automatically store all fields) → **Set Booking Outputs** (Assignment: bookingStartTime, bookingStatus, bookingBikeId set from Get_Booking's fields) → **Set Minutes Until Start** (Assignment: minutesUntilStartOutput = {!minutesUntilStart}) → End

Watch out: This is the exact element that went missing mid-build — the Assignment and/or Formula resource did not actually save the first time, so the canvas silently reverted to Start → Get Booking → Set Booking Outputs → End with no time calculation at all. Always re-open the Flow after saving and visually confirm every element you just added is really on the canvas before moving on.

Note: The org's internal NOW() calculation showed an offset from the expected wall-clock time (roughly 12+ hours), most likely a timezone configuration difference. Rather than chase down the exact Setup → Company Information timezone setting, testing was done empirically: create a test record, run Debug immediately, read the actual minutesUntilStartOutput value, and adjust the test record's time until the desired range (under or over 120) was reached. This is a reasonable workaround for a demo; a production fix would involve aligning the org's default time zone with the operating context.

7. Flow 4: Cancel_Booking

Type: Autolaunched Flow | **Purpose:** Cancels a booking — only ever invoked by the agent when the booking starts in more than 2 hours.

Input variables

- bookingId — Text — Available for input

Output variables

- success — Boolean — Available for output

Canvas logic

Start → **Cancel Rental** (Update Records: "Specify conditions to identify records, and set fields individually" → Object: Rental__c → Condition: Id Equals {!bookingId} → Set Field Values: Status__c = 'Cancelled') → **Set Success** (Assignment: success = True) → End

Note: Use "Specify conditions to identify records, and set fields individually" for Update Records rather than "Use the IDs and all field values from a record or record collection" — the latter option did not reliably expose a Set Field Values editor during this build.

8. Flow 5: Escalate_To_Human

Type: Autolaunched Flow | **Purpose:** The guardrail's safety valve — creates a Case for human review instead of allowing a same-window cancellation.

Input variables

- bookingId — Text — Available for input
- Reason — Text — Available for input (note capital R in this org's final version)

Output variables

- caseId — Text — Available for output

Canvas logic

Start → **Create Escalation Case** (Create Records → Object: Case → Set Field Values manually: Subject = 'Urgent rental change request' (literal text), Description = {!Reason}, Origin = 'Web') → **Set Case Output** (Assignment: caseId = {!Create_Escalation_Case.Id}) → End

Watch out: The Assignment must reference **{!Create_Escalation_Case.Id}** specifically — assigning the whole record ({!Create_Escalation_Case} without .Id) into a Text variable is a type mismatch that can silently misbehave when invoked live by the agent, even though manual Debug runs may tolerate it.

9. Agentforce Agent Structure

Agent Name: Pedal Pro Assistant | **Dedicated user:** EinsteinServiceAgent User (Profile: Einstein Agent User, License: Einstein Agent)

System-level configuration

- Welcome message: "Hi, welcome to Pedal Pro Rentals! I can help you check bike availability, book a rental, or manage an existing booking. What can I help you with today?"
- System instructions: "You are a helpful rental assistant for Pedal Pro Rentals. Help customers check bike availability, create bookings, modify existing rentals, and cancel bookings — following all rental policies."

Subagents

Subagent	Trigger phrases (router)	Purpose
bike availability	"available", "find a bike", "what bikes"	Look up bikes by type/location
bike booking	"book", "rent", "reserve"	Collect details, create the rental via Flow
booking cancellation	"cancel", "cancellation"	Look up booking, apply the 2-hour guardrail policy

Plus the default system subagents kept unchanged: Escalation, Off Topic, Ambiguous Question, and the top-level Agent Router (Start Agent) that welcomes the user and routes to the correct subagent.

10. Final Agent Actions Reference

Every action below is Reference Action Type = **Flow**. Versioned names (v2, v3, v4, v5...) reflect that Agentforce did not reliably pick up input/output changes when a linked Flow's variables changed — the reliable fix each time was to delete the stale action and recreate it fresh from the Flow, rather than edit it in place.

Action (final active name)	Flow	Inputs	Outputs
Check Bike Availability v3	Check_Bike_Availability	Location (String), bikeType (String)	resultMessage (String), bikeCount, bikeListTe
Book Rental v5	Book_Rental_Flow	bikeId, customerId, startDateTimeText, endDateTimeText, totalAmount	confirmationMessage (String), succe
Get Booking Details v4	Get_Booking_Name	bookingId (String)	bookingStartTime, bookingStatus, bookingBil
Cancel Booking v2	Cancel_Booking	bookingId (String)	success (Boolean)
Escalate To Human v2	Escalate_To_Human	Reason (String), bookingId (String)	caseId (String)

Note: Earlier versions (v1 through v4, depending on the action) failed either because of a naming mismatch between the action's declared inputs and the actual Flow variable names (e.g. bike_id vs bikeId), or because the Date/Time type could not be set by Agentforce automatically. Always verify a freshly created action's Inputs/Outputs list matches the Flow's real variable names exactly before attaching it.

11. Final Reasoning Instructions (exact text used)

bike availability subagent

```
Help the customer find available bikes at Pedal Pro Rentals.

Ask the customer for the type of bike they want (mountain, road, city, or e-bike) and their preferred location if not already provided.

Use Check Bike Availability v3 to look up available bikes once you have the bike type and location.

Present the results clearly, including any relevant pricing or details.

After showing results, ask if the customer would like to book one of the available bikes.
```

bike booking subagent

```
Help the customer book a bike rental at Pedal Pro Rentals.

Collect these details before proceeding: a specific Bike record ID, a specific Contact record ID, rental start date/time, and rental end date/time. If the customer only gave a bike type and location, use Check Bike Availability v3 first, then ask which specific bike they want. If the customer gave only a name, ask for their Contact ID directly - do not proceed with a name alone.

Once you have a specific Bike ID, a specific Contact ID, and both dates, you MUST call Book Rental v5 immediately. Do not tell the customer their booking is confirmed unless Book Rental v5 has actually been called and returned success = true. If it returns success = false or an error, tell the customer honestly that the booking could not be completed.

When calling Book Rental v5, pass startDateTimeText and endDateTimeText as plain text exactly as the customer stated or reasonably reformatted, e.g. "2026-09-28 10:00 AM". Do not convert to ISO 8601 format.

After a successful booking (success = true), confirm the booking details to the customer including the confirmation ID.
```

booking cancellation subagent

```
Use Get Booking Details v4 to retrieve the booking and check minutesUntilStartOutput.

Apply the following cancellation policy:
- If the rental starts in MORE than 2 hours (minutesUntilStartOutput > 120): use Cancel Booking v2 to cancel the booking and confirm the cancellation to the customer.
- If the rental starts in 2 HOURS OR LESS (minutesUntilStartOutput <= 120): use Escalate To Human v2 to escalate the request. Inform the customer that per Pedal Pro Rentals' policy, cancellations within 2 hours of the rental start time require review by a team member, and that someone will be in touch shortly.

Do not cancel a booking that starts within 2 hours — always escalate those cases.
```

12. The Guardrail — How It Works

This is the project's required business-rule guardrail: **a rental cannot be cancelled by the AI agent if it starts within 2 hours.** Instead of blocking or refusing silently, the agent creates a Case so a human team member follows up.

Why the logic lives in the Flow, not just the prompt

The original design asked the LLM to reason about a variable called **minutes_until_start** — but no action ever actually produced that variable, so the comparison was against nothing, and the agent defaulted to cancelling everything. The fix moved the actual time math into Salesforce Flow logic (a Formula resource: `(Get_Booking.Start_Date_Time__c - NOW()) * 1440`), which reliably outputs a real number (minutesUntilStartOutput) that the agent's instructions then branch on. This is a good general lesson: **do exact arithmetic and comparisons in Flow, and let the LLM handle language and judgment calls, not precise math.**

Verified behavior

Scenario	minutesUntilStartOutput	Agent behavior	Result
Rental starts in ~87 minutes	87.99	Refuses to cancel, escalates	Real Case created (Case #00001028)
Rental starts in ~814 minutes (~13.5 hrs)	813.89 / 819.83	Proceeds to cancel	Rental__c Status__c set to Cancelled

13. Bug Log — Every Issue Hit and How It Was Fixed

Documented here because this is genuinely useful material for explaining the project in an interview — it shows real debugging, not just following a tutorial.

#	Symptom	Root Cause	Fix
1	Picklist field saved as Multi-Select with wrong API name (Mounting_c)	Deleted, left over text in the Field Name box	Deleted the field and recreated it with the cor
2	Rental__c's Customer field pointed to Account instead of Customer__c	Wrong object selected in the lookup Related picker	Deleted and recreated the field, explicitly selecting the st
3	Booking Flow: successful bookings showed "No Available Branches"	The Not Available branch ran because the array was empty	Debugged the array connection and the Available branches A
4	Agentforce action showed "'X' is not a defined instruction" errors	Stale instructions referenced actions that had been renamed	Edited each instruction to fix (e.g. removed the Rental_ prefix) that al
5	Agentforce action input names didn't match the Flow's variable names	Actions were auto-generated by Agentforce from the Flow, using	Deleted the old actions and re-added them via the Flow Action wizard
6	CheckBikeAvailability action's output variable was never populated	The Flow's "Get Records" result variable was not explicitly	Added an explicit Assignment element to map the Get Records
7	Booking action showed Date/Time Agentforce doesn't support Object type	Agentforce generates input types from Flow Data Types, and	Changed the Flow's booking input to Text, and used a
8	Agent confidently told the customer a booking was complete, but no	The booking was completing, but the Flow was not	Debugged the Flow, identified the missing Create Records
9	Guardrail always cancelled bookings, even when the customer was	The instructions referenced a variable (minutesUntilStart)	Added an explicit step to create an output variable (Get_B
10	A newly added Flow element (the missing Create Records step)	The save did not fully persist the new element	Deleted and re-added the element in the Flow, visually confirmed the canvas sequ
11	Escalation claimed success but no Case was created	Case creation was the last element in the Flow, and	Verified directly via Salesforce reports/SOQL rather than relying on
12	Agent's dedicated user could not execute the Flow or create Cases	The Flow ran as Agentforce Service User, which	Created a dedicated Permission Set ("Agentforce Bike A

14. Permission Set — Agentforce Bike Access

Setup → **Permission Sets** → **New** → Label: Agentforce Bike Access

Object	Permissions granted	Field access
Bike__c	Read, Create, Edit	All fields — Read + Edit access
Rental__c	Read, Create, Edit	All fields — Read + Edit access
Case	Read, Create, Edit	Standard fields used by the Escalate flow

Assigned to: **EinsteinServiceAgent User** (Setup → Users → find the user with the Einstein Agent license → Permission Set Assignments → Edit Assignments → add Agentforce Bike Access → Save).

Watch out: This permission gap was the root cause of the very first major bug (CheckBikeAvailability silently failing when called live, while working fine in manual Debug run as the System Administrator). If a new action or Flow silently fails only when called by the agent but works in Debug, check this first.

15. Verified Test Log

Feature	Test input	Result
Availability	"Do you have a mountain bike available in Jubliee Hills?"	Correct grounded answer after permission fix
Booking	Bike a00g800002GCoObAAL (Ducati), Contact 003g800000kcx[illegible]	Rental created, Bike status flipped to Rente
Cancellation (allowed)	Rental starting ~814 minutes away	Cancelled correctly; Rental Status__c set to Cance
Cancellation (guardrail)	Rental starting ~88 minutes away	Correctly refused; Case #00001028 created by the

16. What's Left To Do

A. Record the demo video (2–3 minutes)

- Beat 1 — Availability: ask about a real Available bike/location combination
- Beat 2 — Booking: provide a real Bike ID + Contact ID (or walk through the natural conversation), then cut to Salesforce to show the new Rental__c record and the Bike__c Status now "Rented"
- Beat 3 — Guardrail: cancel a booking known to start within 2 hours (verify with Flow Debug first), show the agent refuse and escalate, then cut to the new Case record
- Use Loom or a screen recorder; keep it under 3 minutes; set sharing to "Anyone with the link"

B. Screenshots to capture

- Availability answer in the agent chat
- Booking confirmation in the agent chat
- The Rental__c record detail page
- The guardrail refusal message in the agent chat
- The Case record created by the guardrail

C. GitHub repository

- Create a public repo, e.g. agentforce-bike-rental-assistant
- Add the README (template already drafted in an earlier conversation) with the video link and screenshots
- Optional but strong: use Salesforce CLI to retrieve the CustomObject and Flow metadata and include it in a /metadata folder as proof of real configuration

D. Resume / LinkedIn

- Add a one-line project entry near the top of the resume linking to the GitHub repo
- Add the project under LinkedIn's Featured or Projects section
- Consider posting the Loom video natively on LinkedIn for better reach

17. Key Lessons Learned

- **Debug success in Flow Builder does not guarantee success when the agent calls it live.** The same Flow can behave differently depending on the caller's permissions and how input types are passed.
- **Recreate, don't edit, when an Agentforce action's inputs/outputs look stale.** Editing an action in place after its linked Flow changes was unreliable throughout this build; deleting and recreating fresh from "Create Custom Action" consistently fixed it.
- **Do exact math and comparisons inside Flow, not inside agent instructions.** Asking the LLM to reason about a variable no one calculated, or to do exact date arithmetic, is fragile. Calculate it in a Formula resource and hand the agent a ready-made number.
- **An agent that confidently claims success can be wrong.** Instructions should explicitly require checking a real success/failure output before confirming anything to the customer — and the builder should verify against the actual data, not the agent's stated response, especially while testing.
- **Watch for duplicate or leftover instruction lines.** Several bugs traced back to two conflicting versions of the same instruction sitting in the same subagent — always read the full instruction list after each edit, not just the line just changed.
- **Permissions are a common, easy-to-miss root cause.** The agent's dedicated user is a separate identity from the human admin building the agent, and needs its own explicit object/field access via a Permission Set.

End of master reference. This document reflects the exact state of the Pedal Pro Rentals Agentforce project as built and verified during development.